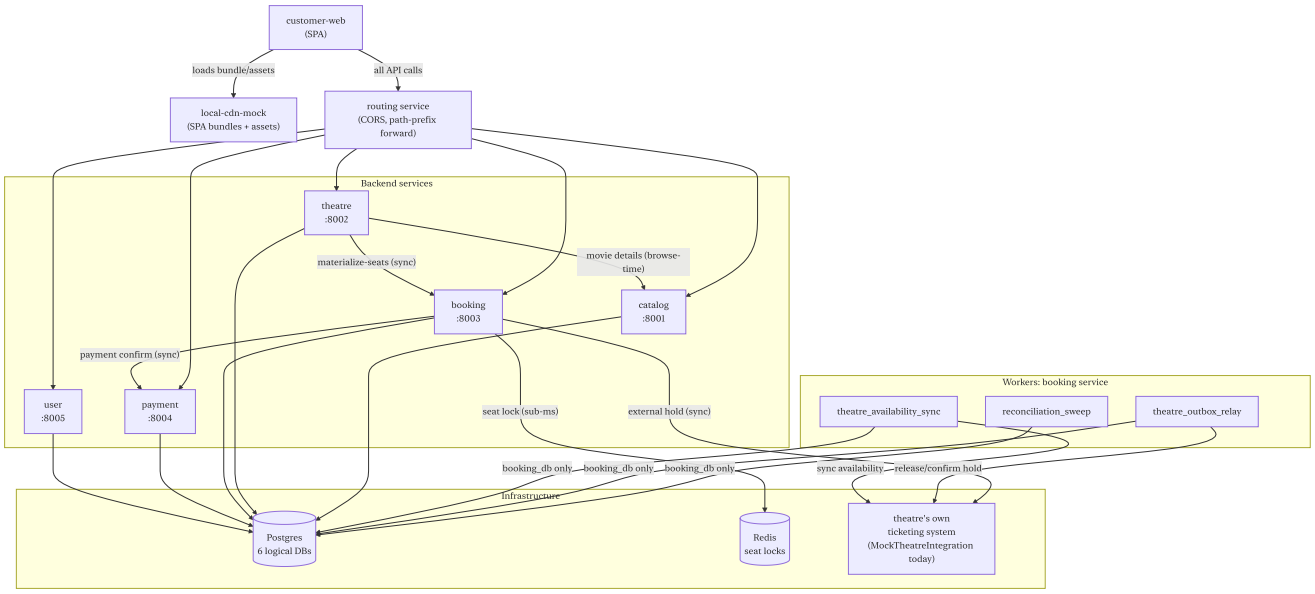
Movie ticket booking platform — comprehensive architecture document

BookMyShow-style aggregator: users browse movies, pick a theatre and showtime, select seats, hold them for 10 minutes, pay, confirm. The same seat is simultaneously bookable via the theatre's own website, other aggregators, and box-office terminals — the platform is an aggregator, not the inventory owner, and the architecture reflects that at every level.

---

1. System topology



Key topology rules:

- Frontend calls routing only — single ingress, same slot a real API gateway fills in production
- Routing is stateless — adds CORS headers, forwards by path prefix, no auth/rate-limiting
- Workers touch `booking_db` only — they never call another service (except `TheatreIntegration`, which is currently mocked in-process)

---

2. Services — detailed breakdown

2.1 Catalog service (`:8001`)

Responsibility: Movie metadata and per-city release windows.

Customer API: `GET /catalog/movies?city=` (only `is_active=true` movies with an active release in that city), `GET /catalog/movies/{id}`

Internal structure (v20):

```
services/catalog/
├── customer/routes.py           # browse-only Appendix A handlers
├── common/db.py + config.py     # shared DB dep, AUTH_ENABLED,
_derive_idempotency_key
└── main.py                      # thin composition root — include_router,
/health
```

Dependencies: None outbound. Theatre service calls it at browse time.

2.2 Theatre service (`:8002`)

Responsibility: Theatre/screen directory, freeform seat-layout authoring with draft lock, showtime scheduling. Triggers seat materialization in booking.

Customer API: `GET /theatre/cities`, `GET /theatre/movies/{id}/showtimes?city=&date=` (one live call to catalog per request — acceptable at browse frequency), `GET /theatre/theatres/{id}`, `GET /theatre/showtimes/{id}`

Internal structure (v20):

```
services/theatre/
├── customer/routes.py
├── common/db.py + config.py     # AUTH_ENABLED lives here, not main.py
(circular import if in main)
└── main.py
```

Outbound calls: `POST /booking/internal/showtimes/{id}/materialize-seats` (synchronous, fail-closed, bounded retry); `GET /catalog/movies/{id}` (one per showtimes-enrichment browse request)

2.3 Booking service (`:8003`)

Responsibility: The core — seat inventory, two-phase lock (Redis + external), booking saga, the three workers, and the Outbox.

Customer API: `GET /booking/showtimes/{id}/seatmap`, `POST /booking/bookings`, `GET /booking/bookings/{id}`, `POST /booking/bookings/{id}/confirm`, `DELETE /booking/bookings/{id}`

Internal API: `POST /booking/internal/showtimes/{id}/materialize-seats` (called by theatre, idempotent via unique constraint)

Internal structure:

```
services/booking/
├── domain/
│   ├── booking.py                  # Booking dataclass, BookingStatus enum,
domain exceptions
│   └── theatre_integration.py      # TheatreIntegration Protocol, HoldResult,
SeatStatus
├── application/
│   └── booking_orchestrator.py     # The saga — no state across calls, owns
nothing
├── adapters/
│   ├── redis_seat_locker.py        # RedisSeatLocker — Lua script, hash-
tagged keys
│   ├── mock_theatre_integration.py  # MockTheatreIntegration —
THEATRE_MOCK_HOLD_MODE env
│   ├── postgres_seat_repository.py  # SHOWTIME_SEAT reads/writes incl.
is_effectively_available
│   ├── postgres_booking_repository.py
│   ├── postgres_outbox_repository.py
│   ├── showtime_meta_repository.py
│   ├── payment_client.py           # HTTP +
CircuitBreaker(PaymentServiceUnavailable)
│   ├── circuit_breaker.py          # Generic closed/open/half-open, shared
│   ├── reconciliation_sweep.py     # Worker 1
│   ├── theatre_outbox_relay.py     # Worker 2
│   └── theatre_availability_sync.py # Worker 3
└── api/main.py
```

Outbound calls: Redis (lock), payment service (confirm), `TheatreIntegration` (hold/confirm/release/sync — mocked)

2.4 Payment service (`:8004`)

Responsibility: Mocked payment — always succeeds. Its value is architectural: it forces booking's confirm path to exercise a real HTTP call, a real circuit breaker, and a real "payment down" failure path, even without a real payment gateway behind it.

API: `POST /payment/payments`, `GET /payment/payments/{id}`

2.5 User service (`:8005`)

Responsibility: Registration, login, JWT issuance. The only service that mints tokens; all others only verify via `shared/auth/auth.py`.

API: `POST /user/auth/register` (409 on duplicate email — see §7 on the idempotency exception), `POST /user/auth/login` → JWT with `{sub: user_id, role: CUSTOMER|ADMIN}`, `GET /user/users/{id}`

Note: Table named `app_user`, not `user` — `user` is a reserved Postgres word; `CREATE TABLE user` is a syntax error.

2.6 Routing service (`:8000`)

Stateless path-prefix forwarder. Adds CORS headers (`allow_origins=["*"]` — acceptable because `AUTH_ENABLED=false` and no cookies/credentials are in use; a real API gateway replaces this slot in production with its own policy). Deliberately does no auth, rate limiting, or request transformation.

2.7 Local CDN mock (`:8006`)

Local-dev-only. Two route groups on one process:

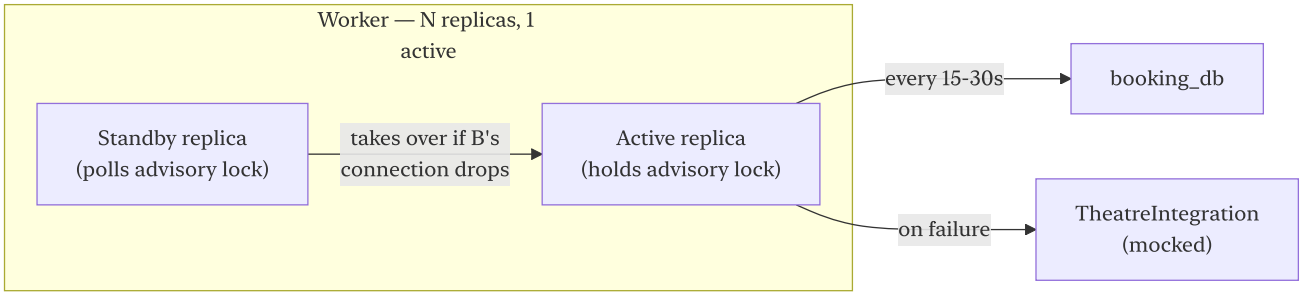
- `/` — SPA bundle served with `html=True` fallback for client-side routing
- `/assets/*` — uploaded images; `assetsDir: 'app-assets'` in Vite config avoids collision with this route; `GET /admin` (no trailing slash) explicitly redirects to `/admin/` (307) before the catch-all mount

Replaced by real static hosting + CDN + object storage in production.

---

3. Workers

All three live in `services/booking/adapters/`. Pattern: **N replicas, exactly one active**, elected via `pg_try_advisory_lock(key)` on a dedicated non-pooled connection, automatic failover within ~one poll interval on connection death.



Worker	Lock key	Cadence
`reconciliation_sweep`	`84236501`	Every 15–30s (`RECONCILIATION_POLL_INTERVAL_SECONDS`)

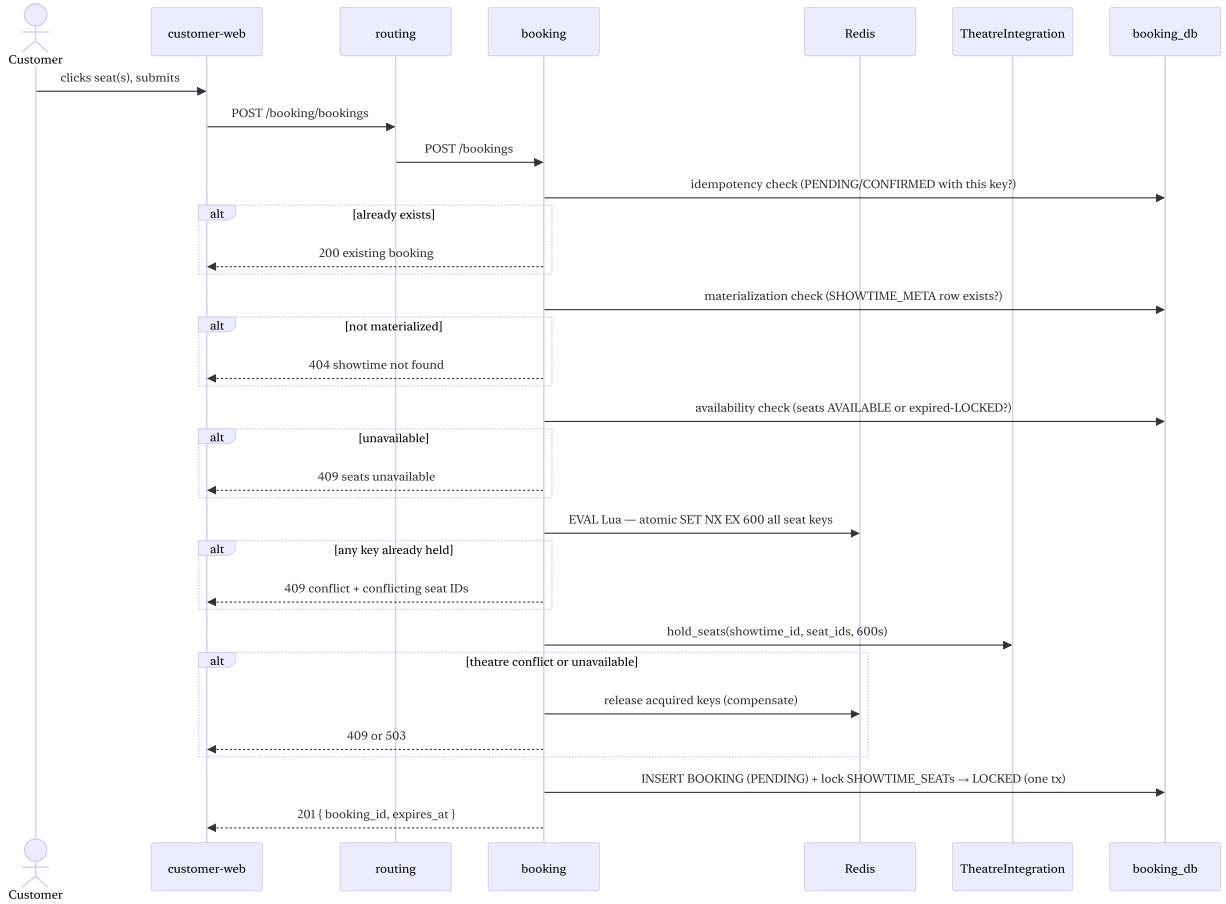
Worker	Lock key	Cadence
`theatre_outbox_relay`	`84236502`	Continuous — polls `pending_theatre_call` every `OUTBOX_RELAY_POLL_INTERVAL_SECONDS` (10s default)
`theatre_availability_sync`	`84236503`	Configurable — `AVAILABILITY_SYNC_POLL_INTERVAL_SECONDS` (60s default for active showtimes)

Why three separate workers: each has an independent failure mode, cadence, and blocked state. A single combined loop would mean one's backlog or bug blocks the others; the advisory lock approach gives each its own independent leadership election.

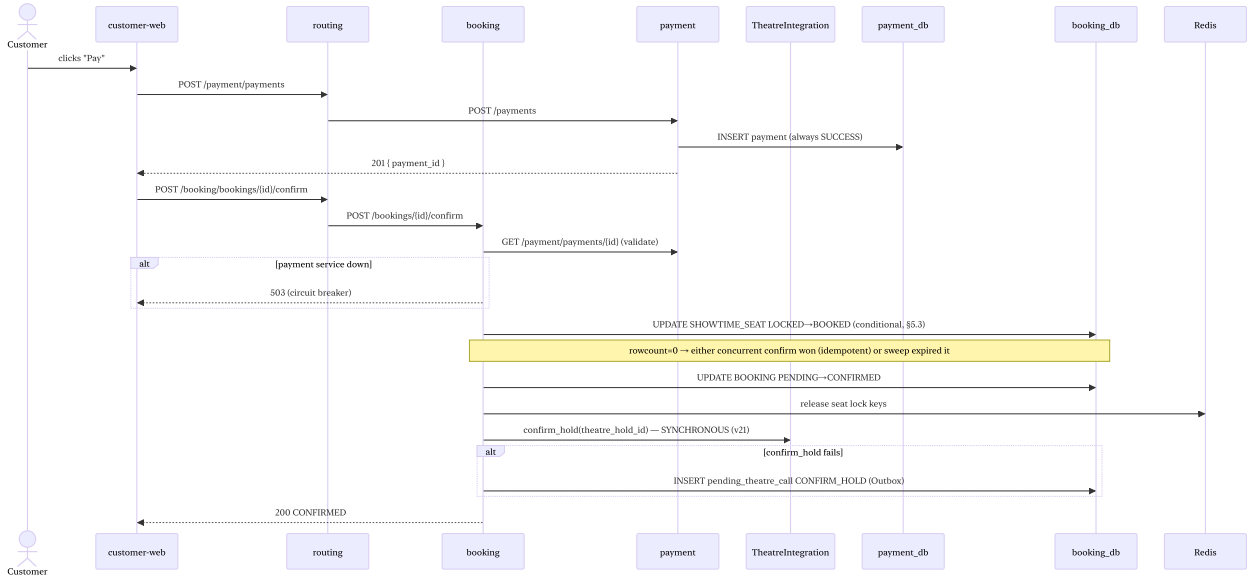
---

4. Sequence diagrams

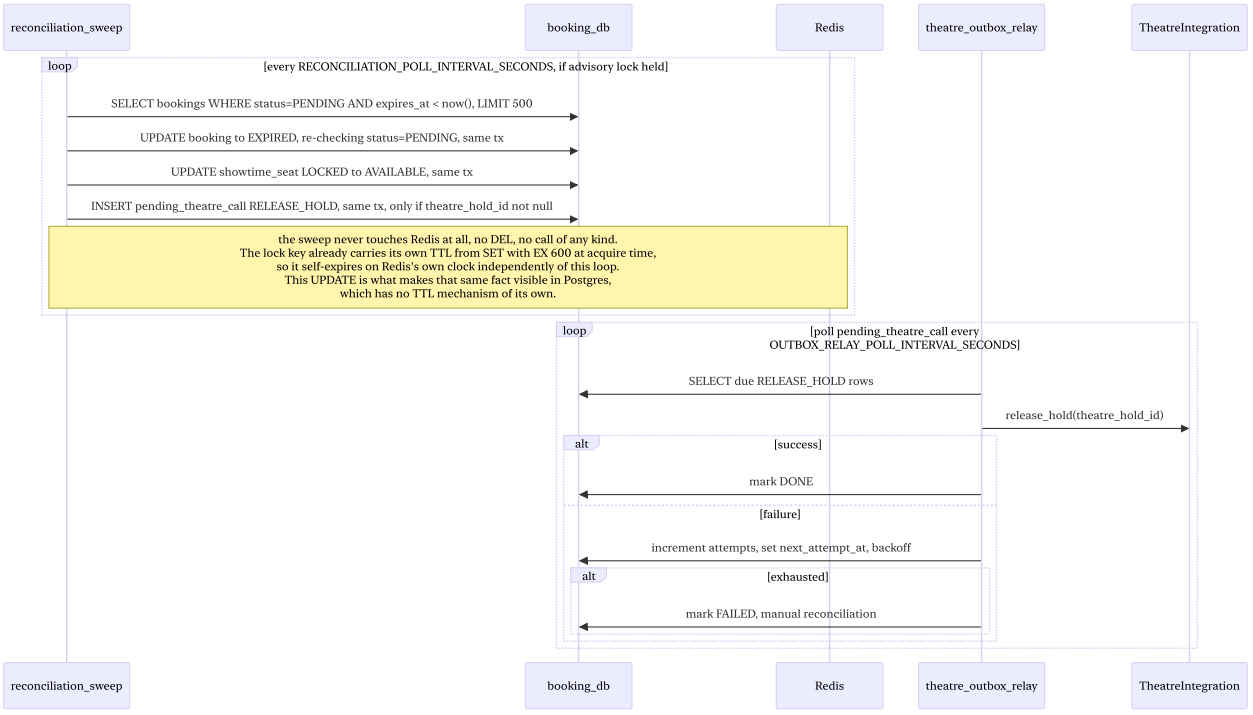
4.1 Seat selection and booking



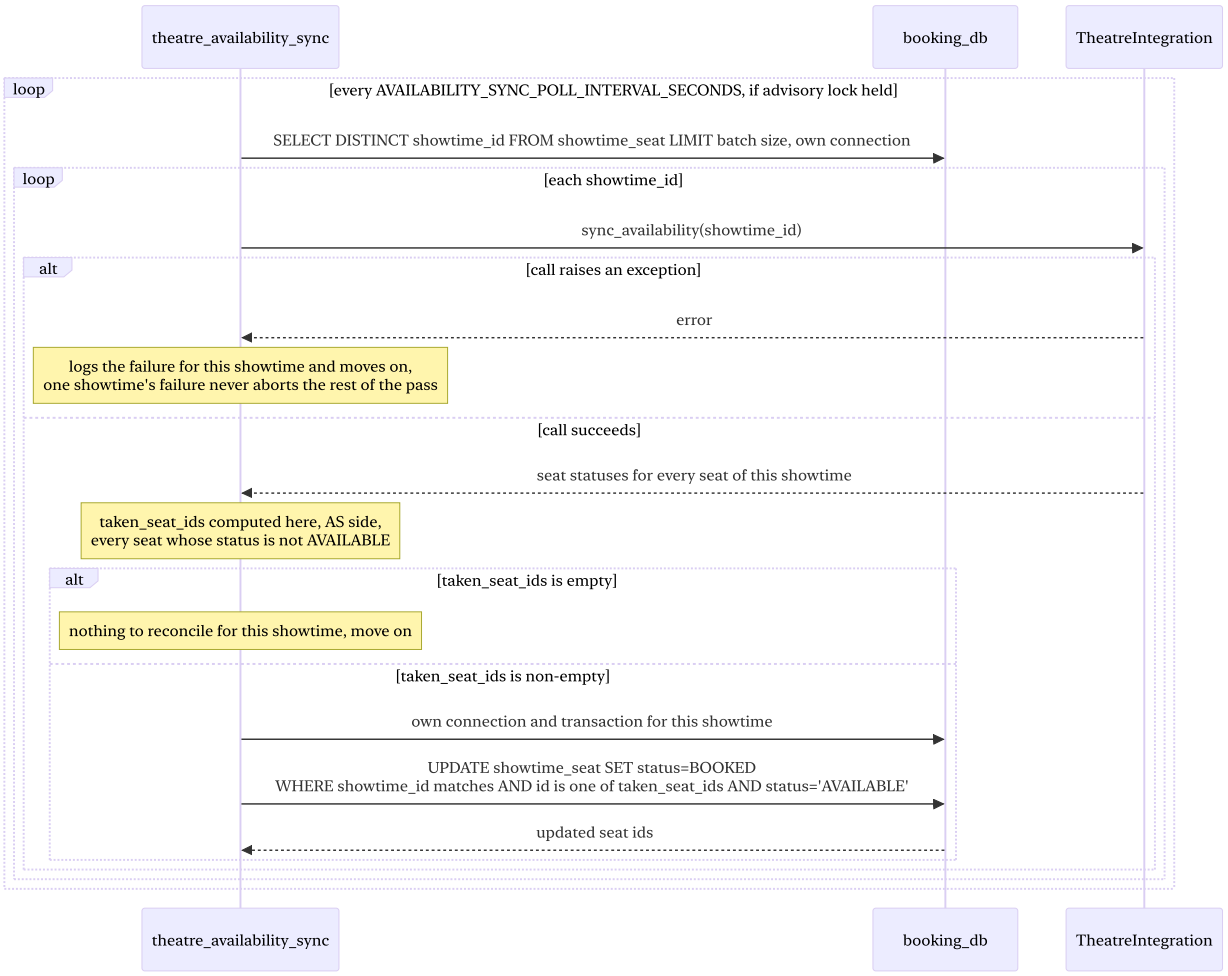
4.2 Payment and confirmation



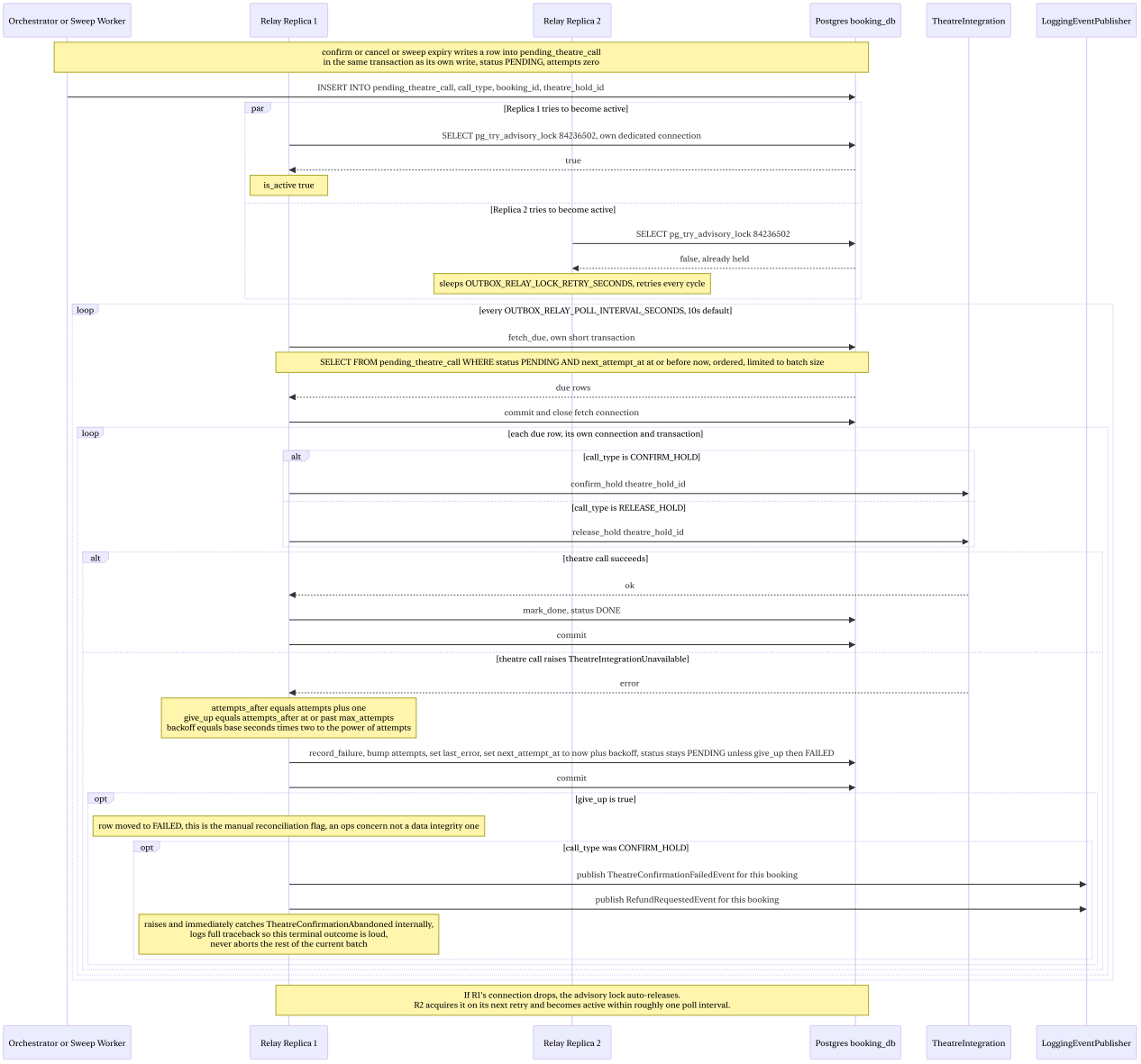
4.3 Hold expiry — sweep worker path



4.4 Theatre availability sync



4.5 Theatre outbox relay



5. Databases — schema and purpose

One Postgres instance locally (six logical databases) → one managed instance per service in production, with async streaming read replicas, PgBouncer connection pooling, Patroni-based HA failover.

`catalog_db`

Table	Key columns	Purpose
`movie`	`id`, `title`, `description`, `duration_min`, `language`, `poster_asset_id`, `is_active`	Source of truth for movie metadata. Soft-delete only — `is_active=false`, never removed, so loose references from other services never "go missing."
`movie_release`	`id`, `movie_id` (loose ref), `city_id` (loose ref to theatre's `CITY`), `release_date`,	Per-city release window. Drives "currently playing" queries — a movie isn't visible to customers in a

Table	Key columns	Purpose
	`planned_end_date`, `actual_end_date`	city until `release_date ≤ today ≤ end_date`.

`theatre_db`

Table	Key columns	Purpose
`city`	`id`, `name`, `state`	Source of truth for cities. Referenced loosely by catalog. Seed-script-only at this phase.
`theatre`	`id`, `city_id` (real FK), `name`, `address`	A physical cinema complex. Real FK to `city` — within the same DB.
`screen`	`id`, `theatre_id` (real FK), `name`	A projection room within a theatre, one seat layout at a time.
`seat_layout`	`id`, `screen_id` (real FK), `name`, `status` (DRAFT/ACTIVE), `locked_by_user_id`, `lock_acquired_at`, `lock_heartbeat_at`	The authored layout for a screen. The three lock columns implement the pessimistic draft lock (§4.6 of the design doc).
`seat_template`	`id`, `seat_layout_id` (real FK), `label`, `position_x`, `position_y`, `seat_type`, `price_multiplier`, `is_active`	One seat in a layout. `position_x` / `position_y` are raw layout units written by the admin canvas's placement tools — normalized per-showtime at materialization time. `price_multiplier × base_price` = the real ticket price. No row/column structure — fully freeform.
`showtime`	`id`, `screen_id` (real FK), `movie_id` (loose ref), `movie_title` (snapshot), `start_time`, `base_price`, `is_high_demand`, `is_active`	A specific screening. Created `is_active=false`; only visible and bookable once explicitly activated. No hard delete.
`theatre_movie_run`	`id`, `theatre_id` (real FK), `movie_id` (loose ref), `start_date`, `end_date`	Optional per-theatre run window — convenience for display/search, not the operational bookability source of truth (that's whether bookable `SHOWTIME` rows exist).

`booking_db`

The most write-sensitive database. Vertically scaled primary for IOPS; `SHOWTIME_SEAT` and `BOOKING` date-partitioned for bounded index sizes and cheap retention drops.

Table	Key columns	Purpose
`showtime_seat`	`id`, `showtime_id` (loose ref), `seat_template_id` (loose ref — for the materialization uniqueness guard), `label`, `position_x`, `position_y`, `seat_type`, `price`, `status` (AVAILABLE/LOCKED/BOOKED), `locked_by_booking_id`, `lock_expires_at`	The bookable for one showti copy, not a ref row per seat p `is_effective = (status='A OR (status=' lock_expires` is the real avai predicate (v13) constraint: `(s seat_templat` prevents dupli materializatio calls. Partial u `(showtime_id seat_templat status='BOOK` structural dou prevention.
`showtime_meta`	`showtime_id` (unique), `movie_title`, `theatre_name`, `screen_name`, `start_time`, `base_price`	A local cache p once at materi Specifically ex seatmap read system's highe read) never ne cross-service c
`booking`	`id`, `idempotency_key`, `user_id` (loose), `showtime_id` (loose), `movie_title`, `seat_labels`, `price_paid`, `status` (PENDING/CONFIRMED/EXPIRED/CANCELLED), `expires_at`, `theatre_hold_id`	The booking r Snapshots eve needs at creati accurate forev of later upstre Idempotency l `sha256(show
`pending_theatre_call`	`id`, `call_type` (CONFIRM_HOLD/RELEASE_HOLD), `booking_id`, `theatre_hold_id`, `status` (PENDING/DONE/FAILED), `attempts`, `next_attempt_at`, `last_error`	The Outbox — same transact booking event it. The relay w and retries ind with bounded backoff.

`payment_db`

Table	Key columns	Purpose
`payment`	`id`, `booking_id` (unique — the natural idempotency key), `amount`, `status`	Mocked. Always `status='SUCCESS'`. The `booking_id` UNIQUE constraint means even a mocked payment can't be created twice for the same booking.

`user_db`

Table	Key columns	Purpose
`app_user`	`id`, `email` (unique), `password_hash`, `role` (CUSTOMER/ADMIN)	Named `app_user` to avoid Postgres's reserved `user` keyword. Passwords are hashed, never stored plaintext. Role is embedded in the JWT claim on login.

`asset_db` (local CDN mock only)

Table	Key columns	Purpose
`asset`	`id`, `filename`, `content_type`, `byte_size`, `storage_path`, `content_hash` (idempotency key)	Metadata for uploaded poster/banner images. Content-addressable — identical bytes uploaded twice reuse the same row. Not part of the production ownership model.

---

6. Redis — usage, key design, and capacity

What Redis stores

Only seat locks. Redis is used for exactly one thing: the within-platform seat lock leg of the two-phase booking lock. It is not a cache, not a session store, not a job queue, not a Pub/Sub bus.

Key design

```
lock:{<showtime_id>}:<seat_id>
```

The `{...}` around `showtime_id` is a **hash tag** — required for correctness on a real Redis Cluster. An atomic multi-key Lua script is only atomic if every key it touches maps to the same hash slot. Without a hash tag, different seat keys would hash to different slots and the `EVAL`

would raise `CROSSSLOT`. With the hash tag, all of one showtime's seat keys always colocate on the same slot — safe for atomic operations. Different showtimes still map to different slots, so cluster-wide load distributes normally across shows.

```
Value: booking attempt correlation ID (UUID)
TTL: 600 seconds (SET NX EX 600)
```

Operations

Operation	Command	When
Acquire all seats atomically	`EVAL <Lua script> N key1..keyN val EX 600`	`select_seats` — all-or-nothing, returns conflicting key IDs on partial failure
Release all seats	`DEL key1 key2 ... keyN`	`confirm` (post-BOOKED), `cancel`, sweep worker
Auto-release	Key expiry (TTL)	Abandoned sessions — no explicit release needed

Lua script (atomic all-or-nothing acquire)

```lua
-- Check all keys first; if any is held, set none
for i=1,#KEYS do
    if redis.call('EXISTS', KEYS[i]) == 1 then
        local conflicts = {}
        for j=1,#KEYS do
            if redis.call('EXISTS', KEYS[j]) == 1 then
                conflicts[#conflicts+1] = j
            end
        end
        return conflicts
    end
end
-- All clear: SET NX EX on every key
for i=1,#KEYS do
    redis.call('SET', KEYS[i], ARGV[1], 'NX', 'EX', ARGV[2])
end
return {}  -- empty = success
```

Read-time reconciliation

A lock key that has expired in Redis is already gone — a new `EVAL` against those keys succeeds immediately. But the `SHOWTIME_SEAT` row may still say `LOCKED` until the sweep worker physically runs. The availability predicate in `postgres_seat_repository.py` handles this:

```
WHERE status = 'AVAILABLE'
   OR (status = 'LOCKED' AND lock_expires_at < now())
```

This makes a seat immediately bookable by a new user the moment the Redis TTL expires, without waiting for the sweep.

Capacity at 2M DAU

Calculation:

- 2M DAU × 12% booking attempt rate = ~250K bookings/day
- Average hold duration before completion or abandonment: ~3 minutes
- Concurrent in-flight bookings at peak: burst ~100–300/sec
- Average seats/booking: 2.5
- **Peak concurrent locked seats: 300 req/sec × 180s average hold × 2.5 seats = ~135,000 active lock keys**
- Per key memory: ~70–80 bytes → **Peak memory: ~10.8 MB** — trivially small for any Redis instance

Hot showtime stampede (the real stress test):

- Thousands of concurrent booking attempts against one showtime's 150–300 seats
- All colocate on one Redis Cluster slot (hash-tagged on `showtime_id`)
- Worst case: 5,000 concurrent requests × 1 Lua eval each = 5,000 ops/sec — ~2.5–5% of one shard's capacity
- **Conclusion:** one Redis shard comfortably handles even the worst-case single-showtime stampede

Redis Cluster configuration for production:

- Minimum: 3 master shards × 1 replica each = 6 nodes
- Replication: asynchronous (acceptable — §5.3's Postgres conditional update is the actual double-booking guarantee regardless of Redis state)
- Failure handling: single-node failure → Redis Cluster promotes replica automatically; client retries with exponential backoff — no Postgres fallback needed or built

---

7. Idempotency

Every create-type endpoint uses the same primitive: `IdempotentWriter.insert_or_get()` → `INSERT ... ON CONFLICT (idempotency_key) DO NOTHING RETURNING *`. One atomic operation — no separate check-then-write window, no shared Redis idempotency store.

The key is **always derived server-side** (SHA-256 hash of identity-defining fields) — never client-supplied.

Entity	Identity fields hashed	Constraint type	Special notes
`MOVIE`	title + duration + language	Plain UNIQUE	
`MOVIE_RELEASE`	movie_id + city_id + release_date	Plain UNIQUE	
`THEATRE`	city_id + name	Plain UNIQUE	
`SCREEN`	theatre_id + name	Plain UNIQUE	
`SHOWTIME`	screen_id + start_time	Plain UNIQUE	
`ASSET`	SHA-256 of file bytes	Plain UNIQUE	Content-addressable — same file uploaded twice reuses the row
`BOOKING`	showtime_id + user_id + sorted seat IDs (comma-joined)	**Partial UNIQUE WHERE status IN ('PENDING','CONFIRMED')**	Partial because the same triple is a legitimately recurring identity — once a booking expires/cancels, the same user should be able to retry the same seats
`PAYMENT`	`booking_id` itself	Natural unique key	One booking → at most one payment; no derived hash needed
`POST /auth/register`	email	**Returns 409, not existing row**	Password isn't part of the key — a conflict can't distinguish retry from a different person claiming the same email
Seat materialization	`(showtime_id, seat_template_id)` per row	Unconditional UNIQUE (not a derived hash)	A retried materialize call produces a clean per-row no-op via the constraint directly

8. Capacity planning — 2M DAU

Traffic type	Daily volume	Sustained peak (req/s)	Burst peak (req/s)
Catalog + showtime browsing	~6M	~500	2,000+ (cache absorbs most)
Seatmap views	~4M	~330	1,500+ (served from booking's own DB, zero cross-service)
Booking creation (select seats)	~250K	~20	100–300
Payment confirmation	~250K	~20	100–300
Hot showtime stampede	—	—	Thousands of requests against 150–300 seats in seconds

Data volume:

Data	Daily growth	Retention
`BOOKING` rows	~250K rows, ~1KB/row → ~250MB/day	7+ years (financial), ~12 months hot, then archived
`PAYMENT` rows	~250K rows → ~100MB/day	Same as bookings
`SHOWTIME_SEAT` rows	~50K screens × ~6 showtimes/day × ~150 seats = **~45M rows/day, ~6.75GB/day**	24–48h post-showtime, then purge
Catalog/theatre metadata	Negligible	Retained indefinitely (soft-deleted, never removed)

Database scaling approach — why vertical, not horizontal:
booking_db` is scaled vertically, not horizontally, for three concrete reasons.

Write throughput is well within single-instance capacity. At peak burst of 300 booking req/s, each request touches ~10 row-level operations (UPDATE ~2.5 `SHOWTIME_SEAT` rows, INSERT 1 `BOOKING` row, SELECT 1 `SHOWTIME_META` row). That is 3,000 row ops/sec — roughly 3–6% of what a mid-range single Postgres instance on NVMe SSD handles. Vertical scaling has enormous headroom before write throughput becomes a concern at 2M DAU.

The hot working set fits on one instance. `SHOWTIME_SEAT` grows at ~6.75GB/day but retains only 24–48h of data, giving a live working set of ~13.5GB. `BOOKING` keeps 12 months hot at ~250MB/day, giving ~90GB. Total hot working set across `booking_db` : ~110GB. A single instance with 256–512GB NVMe SSD and 32–64GB RAM (enough for Postgres's `shared_buffers` plus OS page cache to cover the hot set) handles this comfortably. Date-range partitioning on `SHOWTIME_SEAT` and `BOOKING` keeps per-partition index sizes bounded

regardless of total historical volume — a seven-year-old booking being archived does not make today's booking queries slower.

The booking transaction cannot safely span shards. A single booking creation reads `SHOWTIME_SEAT` (availability check), writes `BOOKING` (INSERT), updates `SHOWTIME_SEAT` (LOCKED status), and conditionally writes `PENDING_THEATRE_CALL` — all in one ACID transaction. The partial unique index `(showtime_id, seat_template_id) WHERE status='BOOKED'` that structurally prevents double-booking relies on single-instance atomicity. Horizontal sharding breaks both of these: rows for the same showtime would need to colocate on the same shard to preserve transaction locality (recovering one guarantee), but then user-history queries (`GET /bookings?user_id=X`) become scatter-gather across all shards, and the sweep worker must poll every shard independently. Enforcing the uniqueness constraint across shards requires either application-level coordination or distributed transactions — both significantly harder to get right under concurrency than Postgres's built-in guarantees. Neither tradeoff is justified until vertical scaling is genuinely exhausted, which the numbers above show it is not.

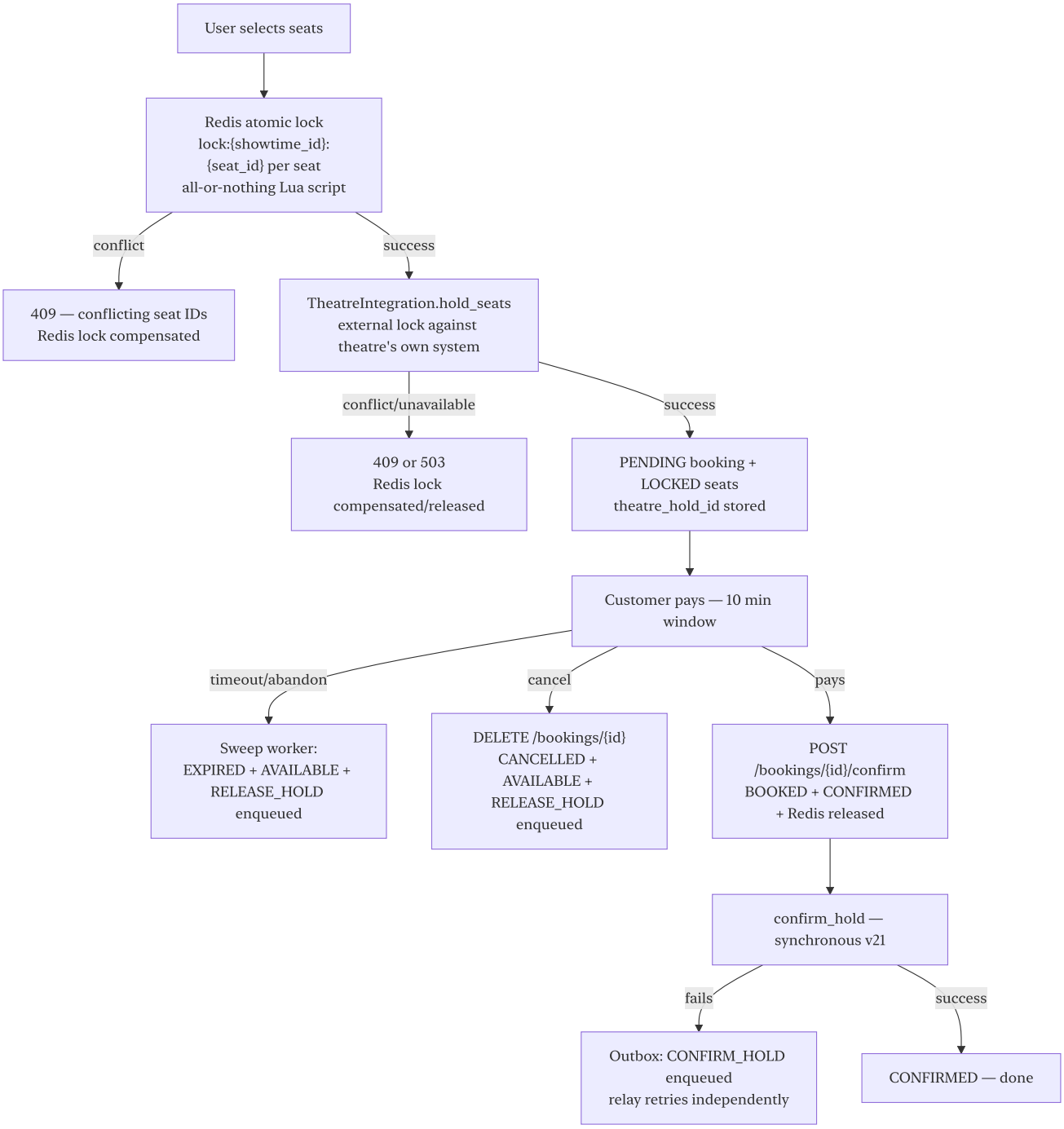
The read path scales differently. `catalog_db` and `theatre_db` serve browsing traffic at ~500–2,000 req/s with low write volume — the right axis here is **horizontal via read replicas**, not vertical, since more replicas directly multiply read capacity without touching write throughput. Browsing queries are explicitly routed to replicas in the repository layer; only writes go to the primary. `booking_db` also has async read replicas serving seatmap reads (~330–1,500 req/s peak); all writes and the confirm/cancel transaction path always go to the primary. The seatmap read tolerates the milliseconds of replica lag because the `is_effectively_available` predicate already handles the stale-lock edge case without needing perfectly fresh data.

PgBouncer sits in front of every service's database. FastAPI instances under load would otherwise exhaust Postgres's native connection limits (default 100) long before hitting any throughput ceiling.

---

9. Seat booking flow — complete detail

The two-phase lock



Why two locks, not one

The Redis lock protects only against concurrent users of this platform. It is invisible to a user booking the same seat via the theatre's own website, another aggregator (Paytm, MakeMyTrip), or a box-office terminal. `TheatreIntegration.hold_seats()` is the cross-channel lock — the theatre's own ticketing system is the actual seat inventory owner. BookMyShow's own `SHOWTIME_SEAT` table is a synchronized shadow copy, not the authoritative source.

Failure scenarios — booking flow only

Scenario	Risk	Mitigation	Tradeoff
Two platform users race for the same seat	Double-booking	Redis atomic multi-key Lua script — all-or-nothing, returns conflicting IDs immediately	One Redis shard per showtime (hash tag) — but single shard comfortably handles even stampede volume
Two requests race past Redis (resharding edge case, Redis bug)	Both believe they hold the lock	Postgres partial unique index `(showtime_id, seat_label) WHERE status='BOOKED'` — structurally impossible to have two BOOKED rows for the same seat	Postgres constraint is slower than Redis; accepted because Redis failure at this level is extremely rare
Same seat booked on the theatre's own site simultaneously	Cross-channel double booking	`TheatreIntegration.hold_seats()` — if they already have it, clean 409, Redis lock released	Adds latency (~100–500ms) on every booking creation; accepted because the guarantee is necessary
Theatre API times out or 5xx on `hold_seats`	Booking hangs or half-completes	Bounded timeout; release Redis lock; return 503 (retryable). Circuit breaker trips after repeated failures	Circuit breaker causes false failures during brief theatre API blips; `HALF_OPEN` probe-and-recover logic handles this
Theatre API confirms hold, then user abandons	Theatre hold lingers	`RELEASE_HOLD` Outbox entry on sweep expiry; relay retries. Theatre typically has its own TTL	Async release means theatre's seat may briefly appear held; not a double-booking risk
`confirm_hold` synchronous attempt fails (payment already taken)	Real cross-channel double-booking risk	Outbox fallback with bounded retries. On exhaustion: `FAILED` , manual reconciliation + `RefundRequestedEvent`	Adds latency to the confirm path; accepted because confirm is the one time-critical moment
Sweep worker crashes mid-batch	Partial expiry	Single transaction per booking — crash leaves the transaction uncommitted, advisory lock releases, standby takes over	Advisory lock re-election adds ~one poll interval of "nobody is sweeping"

Scenario	Risk	Mitigation	Tradeoff
Confirm races sweep for same booking	Sweep expires it just as customer pays	State-gated conditional UPDATE: `WHERE status='PENDING'`. Sweep is sole wall-clock authority (v14) — confirm gates only on state, not the clock	A payment landing just after the countdown displays zero but before the sweep's next pass still completes
Redis full cluster outage	Booking creation fails	Fail fast, clear retryable 503. No Postgres-only lock fallback	Brief unavailability of booking creation only; browsing, viewing, cancellation, sweep all continue normally
Duplicate request / double-click / network retry	Double-booking, double-charge	Partial unique index on `BOOKING.idempotency_key WHERE status IN ('PENDING', 'CONFIRMED')`	Partial index means a new booking attempt for the same seats is possible once the original expires/cancels; intentional

Tradeoffs — booking flow

Tradeoff	What's given up	Why
Redis lock keys hash-tagged on `showtime_id` (one showtime colocates)	Intra-showtime key spreading across nodes	Required for `EVAL` atomicity on a real Cluster (`CROSSSLOT` otherwise); one shard handles even a stampede's burst
`confirm_hold` synchronous, `release_hold` async-only	Asymmetric handling; small added latency on confirm	Confirm is time-critical (theatre TTL can fire during async delay); release isn't
Sweep is sole wall-clock authority (no clock check in `confirm`)	A customer paying just after the countdown can still succeed	More honest — the countdown is the display of a target; the sweep is the actual enforcement
Outbox for `RELEASE_HOLD` / `CONFIRM_HOLD` fallback	New infrastructure (table + worker)	Pattern was referenced for several phases without existing; failure-path tests needed genuine retry behavior
Sync-based shadow inventory for	Staleness window (e.g. 60s) before	Seatmap is the system's highest-volume read (~4M/day); live calls would multiply

Tradeoff	What's given up	Why
seatmap (not live call)	seats taken on other channels appear	that into cross-service load on every seatmap view. Staleness only ever costs a clean 409, never a double-booking
No virtual queue for hot-showtime stampedes	Fast clean 409s instead of smooth queued admission	Correctness holds regardless; single shard absorbs even stampede burst; `SHOWTIME.is_high_demand` reserved in schema for when queuing is worth adding

---

10. Failure scenarios — system-wide

Scenario	Risk	Mitigation
Postgres primary failover	Brief write unavailability during promotion	Patroni/managed-HA automated promotion (10–30s); clients retry with backoff; read replicas continue serving read traffic
`booking_db` replica lag	Stale reads on explicitly-replica-routed queries	Booking-transaction reads always go to the primary; only seatmap/browsing go to replicas
Theatre service down during showtime creation	Showtime creation fails	Fail-closed — no orphan showtimes; retry with bounded backoff before returning error
Payment service down	Booking stays `PENDING` indefinitely	Circuit breaker fails fast (503); booking TTL bounds the impact — sweep eventually expires it
Outbox relay exhausts retries on `CONFIRM_HOLD`	Theatre never learns about a real booking	`FAILED` row + `TheatreConfirmationFailedEvent` + `RefundRequestedEvent` published; manual reconciliation flag
Movie/theatre soft-deleted while bookings reference it	Booking references stale data	Soft-delete never cascades; `BOOKING` snapshots `movie_title` / `seat_labels` / `price_paid` at creation — accurate forever
Materialize-seats retry creates duplicate seat rows	Seat inventory inflated	`UNIQUE (showtime_id, seat_template_id)` on `SHOWTIME_SEAT` — retried rows are per-row no-ops

---

11. Tradeoffs — system-wide

Tradeoff	What's given up	Why
Database-per-service (no cross-DB foreign keys)	Can't enforce referential integrity across services at the DB level	Enables independent scaling and deployment; eliminates cross-service coupling through shared schemas
Soft-delete only, no hard delete	Old records accumulate; DB grows over time	Any hard delete risks leaving another service with a dangling reference; soft-delete means stale IDs always resolve, just to inactive rows
`BOOKING` snapshots `movie_title` / `seat_labels` / `price_paid`	Slight data duplication	The `BOOKING` is a financial record — must remain accurate regardless of what happens upstream. Also makes `GET /bookings/{id}` a single-table query with zero cross-service calls
Freeform seat layout (no row/column)	No free adjacency lookup	Represents any real theatre shape; adjacency not a stated requirement
Server-derived idempotency keys	Two entities with identical identity fields collide into one row	Client carries zero key-generation burden; collision probability is negligible
`MockTheatreIntegration` always succeeds; real adapters are future work	No automated proof against a real POS system's quirks	No real theatre API exists to integrate against yet; the Protocol seam is the zero-orchestrator-change extension point

---

12. Known gaps and future work

Gap	Current state	When/how to address
`AUTH_ENABLED=false` in local dev	No auth in any local request	Phase 10: flip flag, full role × endpoint access-control matrix
At most one ACTIVE layout per screen	Assumed, not DB-enforced	Add a partial unique index `(screen_id) WHERE status='ACTIVE'`

Gap	Current state	When/how to address
No virtual queue	Fast clean 409s under stampede	§16.1 — `SHOWTIME.is_high_demand` schema slot already reserved
`LoggingEventPublisher` no-op	Events published only to logs	Phase 13: real event bus
`MockTheatreIntegration`	No real POS system integration	§16.8 — `THEATRE.integration_type` adapter registry already designed
Real CDN and static hosting	`local-cdn-mock` locally	Phase 13 — config change only, no code change
Real API gateway	Routing service (path-prefix only)	Phase 13 — auth/rate-limiting/WAF
customer-web dark-mode contrast bug	Same latent `index.css` issue as admin-web (fixed Phase 9)	Next frontend pass
Showtime cancellation workflow	Deactivation only; no refund/notification flow	§16.6 — depends on notification system (§16.3) existing first